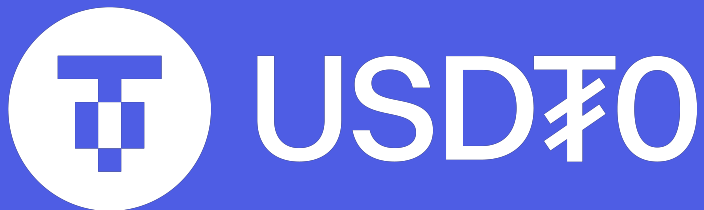


USDT0 Child Token Audit



August 24, 2025

Table of Contents

Table of Contents	2
Summary	3
Scope	4
System Overview	5
Trust Assumptions and Privileged Roles	5
Deployment and Migration Review	7
Polygon Deployment of UChildUSDT0	7
Polygon PoS Bridge Migration	7
Findings	8
Low Severity	10
L-01 Missing Support for Contract Signatures	10
L-02 Missing Support for EIP-3009	10
L-03 Redundant Event Emission	11
L-04 Missing ERC-165 Support	11
L-05 Missing Zero-Address Check	12
Notes & Additional Information	12
N-01 Misleading Documentation	12
N-02 The ecrecover Precompile Is Vulnerable to Signature Malleability	13
N-03 Inconsistent Error Messages	13
N-04 Lack of Security Contact	14
N-05 Inconsistent Order Within Contracts	14
N-06 Function Visibility Overly Permissive	15
N-07 Lack of SPDX License Identifier	15
Conclusion	16

Summary

Type	Stablecoins	Total Issues	12 (3 resolved)
Timeline	From 2025-05-05 To 2025-05-09	Critical Severity Issues	0 (0 resolved)
Languages	Solidity	High Severity Issues	0 (0 resolved)
		Medium Severity Issues	0 (0 resolved)
		Low Severity Issues	5 (2 resolved)
		Notes & Additional Information	7 (1 resolved)

Scope

OpenZeppelin audited the [Everdawn-Labs/pos-portal](#) repository at commit [6a81fd7](#).

In scope were the following files:

```
contracts
├── child/ChildToken/DappTokens
│   ├── UChildUSDT0.sol
│   └── WithBlockedList.sol
```

System Overview

The new USDT0 implementation is aimed to replace the current USDT deployment on Polygon based on the POS-Portal bridge architecture with a modern, omnichain-capable design. At the core of this upgrade is the `UChildUSDT0` contract, which conforms to LayerZero's Omnichain Fungible Token (OFT) standard and extends it by implementing [EIP-7802](#). This enhancement introduces a standardized interface for minting and burning tokens across chains, enabling secure and consistent supply management without relying on wrapped assets or third-party bridges.

USDT0 operates by locking native USDT on Ethereum and minting corresponding USDT0 tokens on other chains such as Polygon. These tokens are fully backed 1:1 and are issued or redeemed based on validated messages transmitted through LayerZero's decentralized messaging network. On Polygon, the `UChildUSDT0` contract is responsible for managing user balances, executing mint and burn operations, and enforcing access control via role-based permissions.

Unlike the previous `UChildERC20` contract, which was tied to a single bridge mechanism and limited administrative capabilities, `UChildUSDT0` serves as a more flexible and secure accounting layer. It receives authenticated instructions from the local OFT contract `OUpgradeable`, which processes incoming cross-chain messages. This separation of responsibilities, message interpretation by the OFT contract and token accounting by `UChildUSDT0`, enhances modularity.

This upgrade supports seamless interoperability across supported blockchains, minimizes liquidity fragmentation, and ensures that the same trusted USDT can move freely between ecosystems, using a single, unified standard.

Trust Assumptions and Privileged Roles

The upgraded USDT0 system on Polygon relies on several key trust assumptions and governance mechanisms to maintain security, compliance, and functional integrity. [EIP-7802](#)

establishes that only explicitly authorized contracts may perform cross-chain mint and burn operations. The system enforces this through tightly scoped role-based access control and contract upgrade governance.

Access within the `UChildUSDt0` contract is defined by the following roles:

- **DEFAULT_ADMIN_ROLE** : The primary administrative authority with full control over the contract. This role can mint and burn tokens, destroy funds of blocked users, override blocklist restrictions during transfers, update the associated OFT contract, and modify the token's name and symbol. It also manages all other roles.
- **BLOCK_ROLE** : Responsible for maintaining the blocklist, including adding or removing users. This role ensures compliance with regulatory requirements or internal risk controls by restricting access to the token.
- **DEPOSITOR_ROLE** : Assigned to the OFT contract on the same chain. This is the only entity authorized to call the `crosschainMint` and `crosschainBurn` functions in compliance with EIP-7802. It acts as the endpoint for cross-chain messages interpreted via LayerZero.
- **Proxy Admin**: Controls the upgrade path for the contract logic via the upgradeable proxy. This role must be strictly protected, as it has the ability to introduce or change the behavior of the system entirely, including modifying role permissions or accounting logic.

Furthermore, the trust model assumes that LayerZero's messaging infrastructure functions correctly and securely, meaning that all cross-chain instructions have been authenticated via decentralized oracle and relayer systems. If this layer is compromised, malicious messages could result in unauthorized token issuance or burning.

Similarly, the security of the system depends on the integrity and operational practices of role holders. The `DEFAULT_ADMIN_ROLE` , `BLOCK_ROLE` , and proxy admin are especially sensitive, and should be governed by secure mechanisms such as multisig wallets and time-locked upgrades.

Successful migration from the old `UChildERC20` to the new `UChildUSDt0` contract also assumes that token balances are preserved without loss or disruption, and that the upgrade is transparent to users and integrators while introducing enhanced functionality and security assurances.

Deployment and Migration Review

This section presents a review of the scripts used to upgrade the USDT0 token implementation and migrate it to a new bridging mechanism on Polygon.

Polygon Deployment of `UChildUSDT0`

The `UChildUSDT0` contract serves as the new implementation for the `UChildERC20Proxy` token contract associated with the Polygon PoS Bridge. Since the `UChildERC20Proxy` contract for USDT was originally deployed within the Polygon bridge ecosystem, its ownership resides with a [Safe wallet](#) controlled by the Matic team.

As a result, deploying and activating the `UChildUSDT0` contract requires close coordination with the Matic team. Their role is to upgrade the proxy so that it points to the new implementation, execute the initializer (`upgradeToUSDT0`) to configure roles and parameters, and finally transfer proxy ownership to the Tether team. This ensures that control of the child token contract transitions over smoothly to the Tether team after the migration.

Polygon PoS Bridge Migration

Historically, USDT transfers between Ethereum Mainnet and Polygon have relied on the Polygon PoS Bridge. Following the upgrade to the `UChildUSDT0` contract, this legacy bridging path will be permanently decommissioned in favor of Tether's new LayerZero-based bridging mechanism, which leverages the Decentralized Verifier Network (DVN).

To enable this transition, the Matic team must disable new USDT deposits on the `RootChainManager` contract while still allowing pending exits for withdrawals initiated prior to the upgrade. At the same time, they must ensure that the correct token balance held in the `ERC20PredicateProxy` contract on L1 is securely transferred to Tether's existing `OAdapterUpgradeable` contract. From that point forward, USDT bridging will be routed exclusively through the OFT adapter using the `send` function, aligning Polygon with the LayerZero-based cross-chain framework.

Findings

We reviewed the relevant deployment and migration scripts in the following two pull requests:

- [Everdawn-Labs/usdt0-ofc-contracts/pull/105](#)
- [maticnetwork/pos-portal/pull/187](#)

Presented below are the relevant findings.

PoS USDT Is Upgraded to the Wrong Implementation

In the `UChildUSDT0Update` script, the `PoS USDT token` contract is upgraded to the wrong implementation. The specified address in the `script's input file` points to a `standard Polygon ERC-20 contract deployed over 4 years ago`.

Consider confirming the latest address of the `UChildUSDT0` implementation before the upgrade.

UChildUSDT0Update Script Upgrades Without Initialization

The `UChildUSDT0Update` script is used to upgrade the current USDT implementation to the new USDT0 token. However, the new implementation contract `exposes an initialization function` that is callable by any address. As such, the script can be executed in two ways:

- Specifying the `updateData` parameter in the input file so that it `upgrades and initializes within the same transaction`
- Not specifying the `updateData` parameter, in which case the script will `only upgrade the contract`, allowing a malicious attacker to take over the contract

Consider combining the `UChildUSDT0Update` and `UpgradeToUSDT0` scripts such that the upgrade can only be executed in combination with the initialization call.

Arbitrary Call Is Overly Permissive When Migrating Token

The `migrateTokens` function currently allows the `MANAGER_ROLE` to perform an `arbitrary call` from the `ERC20Predicate` contract. However, based on the migration requirements, the only intended action is to `transfer USDT held in the ERC20Predicate contract` to a designated receiver. This functionality can be restricted to prevent unnecessary or unsafe arbitrary calls.

Consider updating the `migrateTokens` function to only be able to call the USDT contract's `transfer` function with a specified token amount and recipient.

Unsettled Zero Input Amount for Bridge Fund Migration

The [MigrateBridgeFunds](#) script currently requires the token amount to be manually specified in the [input file](#). At the time of review, this value is set as 0, as an unsettled value. The total amount of USDT that should be migrated is the total supply of USDT on Polygon [UChildERC20Proxy](#) contract right after the upgrade. Any excess USDT amount in the [ERC20Predicate](#) contract will be used to finalize withdrawals initiated before the token upgrade.

Consider updating the unsettled input amount correctly to ensure a successful bridge fund migration.

Low Severity

L-01 Missing Support for Contract Signatures

The current implementation of [the permit function](#) within the `UChildUSD0` contract is designed to only accept Externally Owned Account (EOA) signatures, utilizing traditional ECDSA verification methods. This approach, while effective for individual users operating EOAs, does not accommodate signatures generated by smart contract accounts. As the ecosystem evolves, with the increasing adoption of smart contract wallets and the emergence of standards like [EIP-7702](#), the inability to process contract signatures could represent a significant limitation.

Consider updating the permit function's signature verification process to include support for both EOA and smart contract account signatures. The OpenZeppelin `SignatureChecker` library provides a unified interface for verifying both ECDSA signatures from EOAs and [EIP-1271](#) compliant signatures from smart contract accounts. This change will make the contract more inclusive and adaptable to the evolving landscape of smart account management.

Update: Acknowledged, not resolved. The Everdawn team stated:

| *We tried to stay as close to the current implementation as possible.*

L-02 Missing Support for EIP-3009

The [developer documentation](#) for USD0 indicates support for [EIP-3009](#), which introduces gasless transfers. However, it appears that the current implementation of [the UChildUSD0 contract](#) does not adhere to this standard, as it lacks the `transferWithAuthorization` and `receiveWithAuthorization` functions. This discrepancy between the documentation and the actual contract implementation could lead to confusion and misalignment with user expectations regarding gasless transfer capabilities.

Consider updating the USD0 contract to include the `transferWithAuthorization` and `receiveWithAuthorization` functions, thereby fully supporting EIP-3009 and enhancing the contract's utility and user experience with gasless transfers. Alternatively, consider updating the developer documentation to be inline with the USD0 contract's capabilities.

Update: Acknowledged, not resolved. The Everdawn team stated:

| *We have decided to leave out support for EIP-3009 for this deployment.*

L-03 Redundant Event Emission

In `WithBlocklist.sol`, the `addToBlockedList` and `removeFromBlockedList` functions are used to handle the blocklist for users. However, neither function checks whether the new value differs from the existing one before emitting an event. This might lead to redundant event emissions, thereby incurring unnecessary gas costs. More importantly, these redundant events could mislead other actors or monitor systems, suggesting that an address was added or removed from the blocklist whereas it was not.

Consider verifying whether the address is already present in the blocklist before emitting the event.

Update: Acknowledged, not resolved. The Everdawn team stated:

| *We prefer code simplicity and off-chain management. We do not think this should be verified on-chain.*

L-04 Missing ERC-165 Support

The `UChildUSDT0` contract supports [EIP-7802](#) by implementing the `crosschainMint` and `crosschainBurn` functions to manage cross-chain USDT0 token reserves on supported chains. Contrary to EIP-7802's recommendation, `UChildUSDT0` does not implement the [ERC-165](#) interface, lacking the `supportsInterface` function needed to demonstrate support for the interfaces.

Consider implementing ERC-165 by inheriting `IERC165` and defining `supportsInterface` to return `true` for all the supported interfaces, enhancing compatibility with external contracts.

Update: Resolved in [pull request #18](#) at [commit eeaf85e](#).

L-05 Missing Zero-Address Check

The `UChildUSDT0` contract is missing a zero address check when setting a storage variable. Accidentally setting an address variable to address zero might result in an incorrect configuration of the protocol.

More specifically, the `newAdmin` parameter of the `upgradeToUSDT0` function could benefit from a zero-address check to ensure that the new admin is properly set. Accidentally setting this admin to address zero might result in the contract not being able to perform admin capabilities without updating the implementation address.

Consider adding a zero address check to the `newAdmin` parameter in order to avoid accidental misconfigurations.

Update: Resolved in [pull request #18](#) at [commit b6aeae2](#).

Notes & Additional Information

N-01 Misleading Documentation

The [inline documentation](#) for the `executeMetaTransaction` function of the `NativeMetaTransaction` contract inaccurately states that both the `userAddress` and `relayer` address are appended to the calldata for extraction, whereas the implementation only appends the `userAddress`. This discrepancy can lead to misunderstandings about how the contract processes these addresses, especially since `ContextMixin`, which is expected to interact with this data, does not require the `relayer` address.

The primary impact of this misleading documentation is potential confusion among developers and auditors, who may have incorrect expectations about the contract's functionality. Although this does not affect the contract's operation, since only the `userAddress` is needed and correctly handled, the inaccurate comment could mislead individuals reviewing or interacting with the code.

Consider updating the inline documentation to accurately describe the actual implementation. This correction will align the documentation with the code's functionality and ensure clarity for all parties reviewing the contract.

Update: Acknowledged, not resolved.

N-02 The `ecrecover` Precompile Is Vulnerable to Signature Malleability

The [use of `ecrecover`](#) for signature verification in the permit function exposes the system to potential signature malleability attacks, where a signature could be altered without access to the private key. However, the implementation mitigates the risk of replay attacks by incorporating nonces, ensuring that each action is unique and cannot be executed more than once. While this significantly reduces the potential impact of such vulnerabilities, it does not eliminate the underlying risk associated with signature malleability.

The primary impact of relying on `ecrecover` is the theoretical risk of signature malleability, which could allow attackers to manipulate signatures in a way that the original signer did not intend. However, in this case, the use of nonces effectively prevents replay attacks.

To enhance security and address the signature malleability issue more comprehensively, consider replacing the use of `ecrecover` with OpenZeppelin's `SignatureChecker` library. This library offers a robust solution for signature verification, supporting both ECDSA signatures from EOAs and [EIP-1271](#) signatures from smart contracts. Adopting this library would not only mitigate the risk of signature malleability but also align the contract with best practices regarding support for smart account wallets.

Update: Acknowledged, not resolved.

N-03 Inconsistent Error Messages

The [UChildUSDt0 contract](#) is inconsistent in the naming conventions used within the error messages. Specifically, some error messages refer to `TetherToken`, while others use `UChildUSDt0`. This inconsistency can lead to confusion for developers and users interacting with the contract, as it may not be immediately clear that both names refer to the same entity within the contracts.

Consider reviewing all error messages within the contract to identify any discrepancies in naming conventions and updating the error messages to reflect a consistent naming approach.

Update: Resolved in [pull request #21](#) at [commit 3de98b1](#).

N-04 Lack of Security Contact

Providing a specific security contact (such as an email or ENS name) within a smart contract significantly simplifies the process for individuals to communicate if they identify a vulnerability in the code. This practice is quite beneficial as it permits the code owners to dictate the communication channel for vulnerability disclosure, eliminating the risk of miscommunication or failure to report due to a lack of knowledge on how to do so. In addition, if the contract incorporates third-party libraries and a bug surfaces in those, it becomes easier for their maintainers to contact the appropriate person about the problem and provide mitigation instructions.

More specifically, the [UChildUSDT0 contract](#) does not have a security contact.

Consider adding a NatSpec comment containing a security contact above each contract definition. Using the [@custom:security-contact](#) convention is recommended as it has been adopted by the [OpenZeppelin Wizard](#) and the [ethereum-lists](#).

Update: *Acknowledged, not resolved.*

N-05 Inconsistent Order Within Contracts

Both the [UChildUSDT0](#) and [WithBlockedList](#) contracts deviate from the Solidity Style Guide due to having an inconsistent ordering of functions and events:

- The [UChildUSDT0](#) contract is mixing [external](#) with [public](#) and [internal](#) functions.
- Both the [UChildUSDT0](#) and [WithBlockedList](#) contracts define events at the bottom of the contract.

To improve the project's overall legibility, consider standardizing ordering throughout the codebase as recommended by the [Solidity Style Guide \(Order of functions\)](#).

Update: *Acknowledged, not resolved.*

N-06 Function Visibility Overly Permissive

Throughout the codebase, multiple functions with unnecessarily permissive visibility were identified. The following functions with `public` visibility could be limited to `external`:

- In `UChildUSDT0.sol`, the `upgradeToUSDT0`, `transferFrom`, `mint`, `redeem`, and `destroyBlockedFunds` functions
- In `WithBlockedList.sol`, the `addToBlockedList` and `removeFromBlockedList` functions

To better convey the intended use of functions and to potentially realize some additional gas savings, consider changing a function's visibility to be only as permissive as required.

Update: *Acknowledged, not resolved.*

N-07 Lack of SPDX License Identifier

The `UChildUSDT0.sol` file lacks an SPDX license identifier.

To avoid legal issues regarding copyright and follow best practices, consider adding SPDX license identifiers to files as suggested by the [Solidity documentation](#).

Update: *Acknowledged, not resolved.*

Conclusion

OpenZeppelin audited the new USDT0 architecture upgrade for the USDT token on Polygon, which adopts LayerZero's OFT standard and introduces support for EIP-7802. This upgrade transitions the system away from the legacy POS-Portal bridge, enabling standardized and secure cross-chain mint and burn operations. The audit focused on the [UChildUSDT0](#) contract and its role in managing accounting, access control, and EIP-7802 compliance.

The review identified a few areas for improvement, including missing support for standards such as EIP-1271, EIP-165, and EIP-3009, an issue involving the blocklist logic, and minor suggestions to improve code clarity and robustness. The Everdawn Labs team was responsive and collaborative throughout the audit process, promptly addressing questions and providing clarifications where needed.